TETRIS GAME

Game Development Using Python Tkinter

A beginner-friendly GUI game project featuring falling geometric blocks, keyboard controls, line clearing, score tracking, pause/resume, restart, and exit options.

Name: Sana Saeed S **Roll no.:** 251701049 **Dept:** CSD, II Year

PROJECT REPORT

1. AIM:

To design and develop an interactive Tetris game using Python Tkinter in which players arrange falling geometric blocks, complete horizontal lines, earn points, and avoid filling the game board to the top

2. SOFTWARE REQUIREMENTS:

- **Programming language:** Python
- **GUI library:** Tkinter (included with most standard Python installations)
- **Editor:** Python IDLE or any Python editor
- **Additional packages:** None required

3. PROCEDURE:

1. Import the tkinter and random modules.
2. Create the main game window and Canvas.
3. Create a grid of 10 columns and 20 rows.
4. Define the seven Tetris shapes using two-dimensional lists.
5. Generate random shapes at the top of the board.
6. Use arrow keys to move and rotate the falling shapes.
7. Implement automatic downward movement using Tkinter's after() method.
8. Detect collisions with the walls, floor, and existing blocks.
9. Remove completed rows and update the score.
10. Display Game Over when a new shape can no longer be placed.
11. Provide restart, pause/resume, and exit controls.
12. Run the program and test movement, rotation, scoring, and game-over features.

4. PROGRAM CODE

Save the following code as tetris_game.py and run it using Python IDLE (F5).

```
import tkinter as tk
import random

CELL = 30
COLS = 10
ROWS = 20
WIDTH = CELL * COLS
HEIGHT = CELL * ROWS
DELAY = 450

SHAPES = [
    ([[1, 1, 1, 1]], "#22D3EE"),
    ([[1, 1], [1, 1]], "#FACC15"),
    ([[0, 1, 0], [1, 1, 1]], "#C084FC"),
    ([[1, 0, 0], [1, 1, 1]], "#FB923C"),
    ([[0, 0, 1], [1, 1, 1]], "#60A5FA"),
    ([[1, 1, 0], [0, 1, 1]], "#4ADE80"),
    ([[0, 1, 1], [1, 1, 0]], "#F87171")
]


class TetrisGame:
    def __init__(self, root):
        self.root = root
        self.root.title("TETRIS GAME")
        self.root.resizable(False, False)
        self.root.configure(bg="#111827")

        tk.Label(
            root, text="TETRIS",
            font=("Arial Black", 24),
            bg="#111827", fg="#FACC15"
        ).pack(pady=8)

        self.score_label = tk.Label(
            root, text="SCORE: 0",
            font=("Arial", 14, "bold"),
            bg="#111827", fg="white"
        )
        self.score_label.pack()

        self.canvas = tk.Canvas(
            root, width=WIDTH, height=HEIGHT,
            bg="#0F172A",
            highlightthickness=2,
            highlightbackground="#64748B"
        )
        self.canvas.pack(padx=12, pady=10)

        self.status = tk.Label(
            root,
            text="Arrow keys: Move | Up: Rotate | Space: Drop",
            font=("Arial", 9),
            bg="#111827", fg="#CBD5E1"
```

```
        )
        self.status.pack(pady=4)

        buttons = tk.Frame(root, bg="#111827")
        buttons.pack(pady=8)

        tk.Button(
            buttons, text="RESTART",
            command=self.restart, bg="#22C55E",
            font=("Arial", 10, "bold")
        ).pack(side="left", padx=5)

        tk.Button(
            buttons, text="PAUSE / RESUME",
            command=self.toggle_pause, bg="#38BDF8",
            font=("Arial", 10, "bold")
        ).pack(side="left", padx=5)

        tk.Button(
            buttons, text="EXIT",
            command=root.destroy, bg="#EF4444",
            fg="white", font=("Arial", 10, "bold")
        ).pack(side="left", padx=5)

        root.bind("<Left>", lambda e: self.move(-1, 0))
        root.bind("<Right>", lambda e: self.move(1, 0))
        root.bind("<Down>", lambda e: self.move(0, 1))
        root.bind("<Up>", lambda e: self.rotate())
        root.bind("<space>", lambda e: self.hard_drop())
        root.bind("p", lambda e: self.toggle_pause())
        root.bind("r", lambda e: self.restart())

        self.timer = None
        self.restart()

    def restart(self):
        if self.timer is not None:
            self.root.after_cancel(self.timer)
            self.timer = None

        self.board = [
            [None for _ in range(COLS)] for _ in range(ROWS)
        ]
        self.score = 0
        self.running = True
        self.paused = False
        self.score_label.config(text="SCORE: 0")
        self.status.config(
            text="Arrow keys: Move | Up: Rotate | Space: Drop"
        )

        self.spawn_piece()
        self.draw()
        self.schedule()

    def spawn_piece(self):
        shape, color = random.choice(SHAPES)
        self.piece = [row[:] for row in shape]
        self.color = color
        self.x = COLS // 2 - len(self.piece[0]) // 2
```

```
    self.y = 0

    if not self.valid(self.x, self.y, self.piece):
        self.finish("GAME OVER")

def valid(self, x, y, shape):
    for r, row in enumerate(shape):
        for c, value in enumerate(row):
            if not value:
                continue
            nx, ny = x + c, y + r

            if nx < 0 or nx >= COLS or ny >= ROWS:
                return False
            if ny >= 0 and self.board[ny][nx] is not None:
                return False
    return True

def move(self, dx, dy):
    if not self.running or self.paused:
        return

    if self.valid(self.x + dx, self.y + dy, self.piece):
        self.x += dx
        self.y += dy
        self.draw()
    elif dy == 1:
        self.lock_piece()

def rotate(self):
    if not self.running or self.paused:
        return

    rotated = [list(row) for row in zip(*self.piece[::-1])]

    if self.valid(self.x, self.y, rotated):
        self.piece = rotated
    elif self.valid(self.x - 1, self.y, rotated):
        self.x -= 1
        self.piece = rotated
    self.draw()

def hard_drop(self):
    if not self.running or self.paused:
        return

    while self.valid(self.x, self.y + 1, self.piece):
        self.y += 1
    self.lock_piece()

def lock_piece(self):
    if not self.running:
        return

    for r, row in enumerate(self.piece):
        for c, value in enumerate(row):
            if value:
                nx, ny = self.x + c, self.y + r
                if 0 <= ny < ROWS:
                    self.board[ny][nx] = self.color
```

```
    remaining = [
        row for row in self.board
        if any(cell is None for cell in row)
    ]
    cleared = ROWS - len(remaining)
    self.board = (
        [[None for _ in range(COLS)] for _ in range(cleared)]
        + remaining
    )

    points = {1: 100, 2: 300, 3: 500, 4: 800}
    self.score += points.get(cleared, 0)
    self.score_label.config(text=f"SCORE: {self.score}")

    self.spawn_piece()
    self.draw()

def draw(self):
    self.canvas.delete("all")

    for r in range(ROWS):
        for c in range(COLS):
            color = self.board[r][c]
            if color:
                self.draw_cell(c, r, color)

    if self.running:
        for r, row in enumerate(self.piece):
            for c, value in enumerate(row):
                if value:
                    self.draw_cell(
                        self.x + c, self.y + r, self.color
                    )

    for c in range(COLS + 1):
        self.canvas.create_line(
            c * CELL, 0, c * CELL, HEIGHT, fill="#1E293B"
        )
    for r in range(ROWS + 1):
        self.canvas.create_line(
            0, r * CELL, WIDTH, r * CELL, fill="#1E293B"
        )

    if not self.running:
        self.canvas.create_rectangle(
            20, HEIGHT // 2 - 50,
            WIDTH - 20, HEIGHT // 2 + 50,
            fill="#111827", outline="#38BDF8", width=2
        )
        self.canvas.create_text(
            WIDTH // 2, HEIGHT // 2 - 15,
            text="GAME OVER",
            font=("Arial Black", 22), fill="#F87171"
        )
        self.canvas.create_text(
            WIDTH // 2, HEIGHT // 2 + 20,
            text=f"Final Score: {self.score}",
            font=("Arial", 15, "bold"), fill="white"
        )
```

```
    def draw_cell(self, c, r, color):
        x1, y1 = c * CELL, r * CELL
        self.canvas.create_rectangle(
            x1 + 1, y1 + 1, x1 + CELL - 1, y1 + CELL - 1,
            fill=color, outline="#0F172A"
        )

    def schedule(self):
        if self.running and self.timer is None:
            self.timer = self.root.after(DELAY, self.tick)

    def tick(self):
        self.timer = None
        if not self.running:
            return

        if not self.paused:
            if self.valid(self.x, self.y + 1, self.piece):
                self.y += 1
            else:
                self.lock_piece()
            self.draw()

        self.schedule()

    def toggle_pause(self):
        if not self.running:
            return

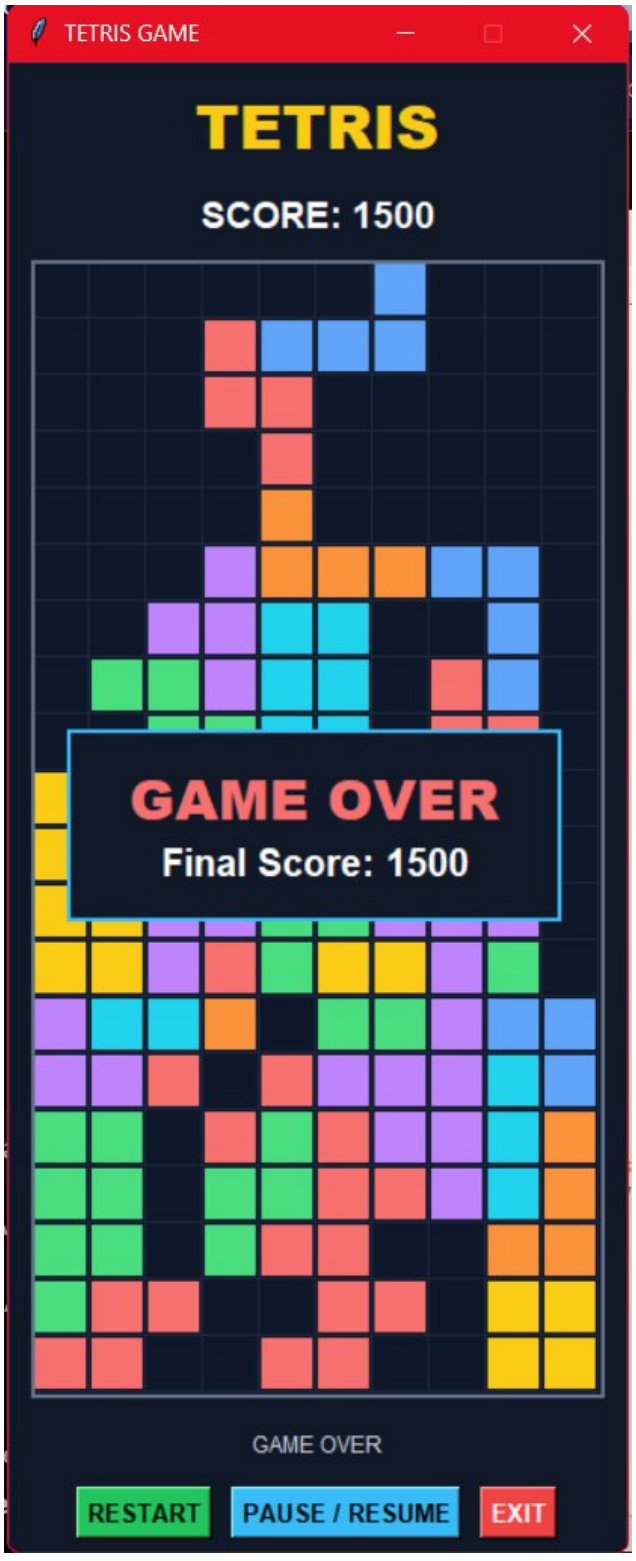
        self.paused = not self.paused
        if self.paused:
            self.status.config(
                text="GAME PAUSED - Press P or Resume"
            )
        else:
            self.status.config(
                text="Arrow keys: Move | Up: Rotate | Space: Drop"
            )

    def finish(self, message):
        self.running = False
        self.status.config(text=message)
        self.draw()

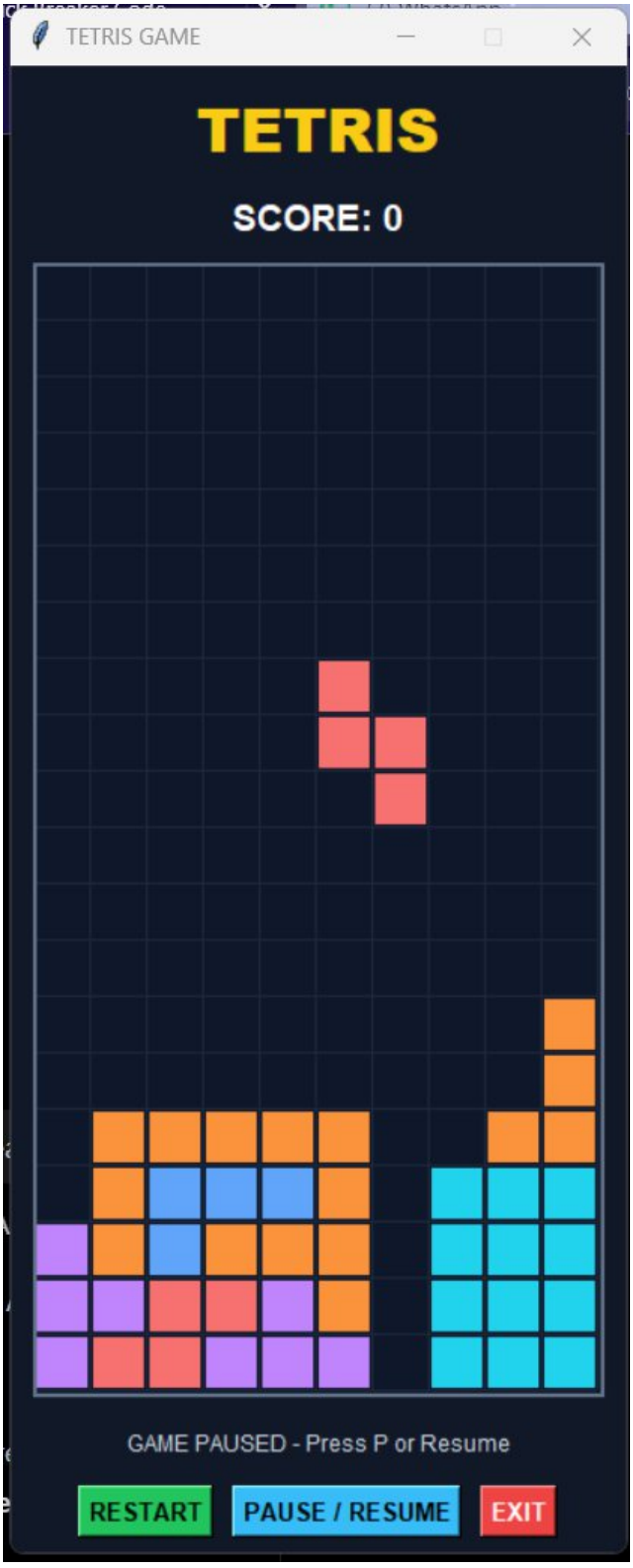

root = tk.Tk()
game = TetrisGame(root)
root.mainloop()
```

5. OUTPUT

The following screenshots were provided and are included as the observed output of the game.



Screenshot 1: Game Over screen showing a final score of 1500.



Screenshot 2: Tetris game paused, showing the board and current score.

6. CONTROLS

Key / Button	Function
Left Arrow	Move the falling block left
Right Arrow	Move the falling block right
Down Arrow	Move the falling block down
Up Arrow	Rotate the falling block
Spacebar	Drop the block instantly
P / Pause-Resume	Pause or resume the game
R / Restart	Restart the game
Exit	Close the application

7. RESULT

Thus, an interactive Tetris game was developed using Python Tkinter. The program demonstrates GUI design, random shape generation, keyboard controls, automatic animation, collision detection, line clearing, score calculation, and game-state management. The supplied screenshots document the game-over and paused states.